

Day 3: From trained model to microcontroller

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 5, 2026

- ▶ A trained model is a file on a laptop. A microcontroller has no Python, no file system for the model, and no operating system.
- ▶ You must change the format of the model, and you need a runtime that fits in kilobytes of memory.
- ▶ In the lab you train a motion classifier on your Day 2 dataset, convert it, and run it on the XIAOML Kit.
- ▶ You measure the arena size, the flash use, and the latency. Day 4 makes these three numbers smaller.

After this day you can:

- ▶ Explain the TensorFlow Lite Micro interpreter, the operator resolver, and the tensor arena.
- ▶ Convert a Keras model and a PyTorch model to the LiteRT format.
- ▶ Extract spectral features from a time series.
- ▶ Deploy a model on a microcontroller and measure arena size, flash use, and latency.

Part 1 Deployment formats and conversion

Exercise: list the operators and the tensor types of a LiteRT file in Netron

Part 2 TensorFlow Lite Micro internals

Exercise: estimate the arena size of a small model

Part 3 Features for time series

Exercise: compare the input size of raw windows and of spectral features

	Training, on the laptop	Inference, on the XIAO
Language	Python	C++
Framework	Keras or PyTorch: hundreds of megabytes	A runtime of some tens of kilobytes
Model	A file on the disk	A constant array in the flash
Memory	Gigabytes, from the operating system	One fixed block of RAM
Operations	All operations of the framework, with training	Only the operations of this model, inference only

- ▶ You cannot copy a training framework to a microcontroller.
- ▶ You change the model into a format that a small runtime can read. This step is the conversion.

Key point: The format of the model and the runtime on the device are one decision. You make it before you design the model.

The conversion pipeline of today



- ▶ **LiteRT** is the new name of TensorFlow Lite. The file extension is still .tflite.
- ▶ **TensorFlow Lite Micro** is the runtime for microcontrollers. It reads the same file format. Part 2 explains it.
- ▶ The C array is the file, byte for byte. The compiler puts it in the flash with the program.

Three runtimes, three device classes

	TensorFlow	LiteRT	TensorFlow Lite Micro
Training	Yes	Limited	No
Inference	Yes	Yes	Yes
Operators	All	Inference operators	The operators that the model selects
Operating system	Necessary	Necessary	Not necessary
Model access	File system	File or mapped memory	Array in the program
Typical device	Server, laptop	Phone, Raspberry Pi	Microcontroller

Key point: Each step to a smaller device removes a part of the software: first the training, then the operating system, then all operators that the model does not use.

Source: *Machine Learning Systems*, Volume I, chapter 7.

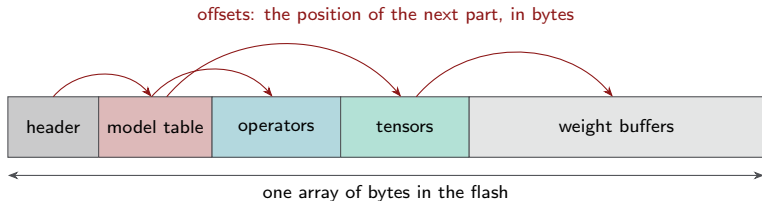
In the file

- ▶ The graph: the operators and their order.
- ▶ The tensors: shape and data type of each input, output, and weight.
- ▶ The weights: the trained values.
- ▶ The parameters of the quantization (Day 4).

Not in the file

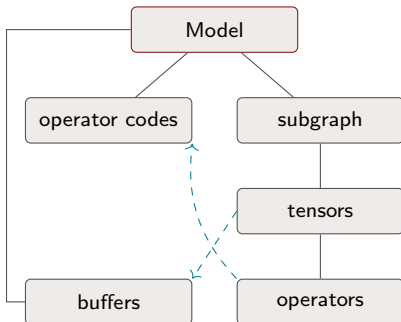
- ▶ The Python code of the model.
 - ▶ The optimizer and the loss function.
 - ▶ The gradients.
 - ▶ Layers that work only in training, for example dropout.
-
- ▶ A deployment format describes only the forward pass.
 - ▶ The runtime needs no knowledge of the framework that trained the model.

FlatBuffers: read the file with no parsing step



- ▶ FlatBuffers is a storage format. Each part holds the offset of the parts that it uses.
- ▶ The runtime reads a value at its position in the array. It makes no copy and builds no objects.
- ▶ So the weights stay in the flash. The RAM holds only the activations.

Key point: A format that needs a parsing step makes a second copy of the model in RAM. A microcontroller has no RAM for that copy.



Operator codes The list of the operator types that the model uses.

Tensors Shape, data type, and name. A weight tensor names its buffer.

Operators One entry for each step: the type, the input tensors, and the output tensors.

Buffers The bytes of the weights.

Key point: The list of the operator codes says which kernels the device needs. A small model uses three or four operator types.

```
import keras
import tensorflow as tf

model = keras.Sequential([
    keras.Input(shape=(300,)),          # 2 s at 50 Hz, 3 axes
    keras.layers.Dense(20, activation="relu"),
    keras.layers.Dense(10, activation="relu"),
    keras.layers.Dense(4, activation="softmax"),
])
# ... train the model ...

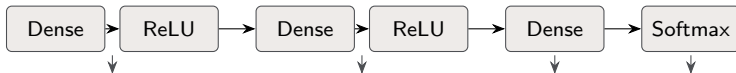
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()      # bytes of the LiteRT file
with open("model.tflite", "wb") as handle:
    handle.write(tflite_model)
```

- ▶ The model has 6274 parameters. With float32, the weights need $6274 \times 4 = 25\,096$ bytes.
- ▶ The LiteRT file has 26 732 bytes: the weights, and 1636 bytes for the graph and the tensor descriptions.

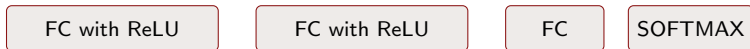
Source: File size: result of the converter of TensorFlow 2.21.0 for this model.

What the converter does

Keras: 6 steps



LiteRT: 4 operators (FC = FULLY_CONNECTED)



- ▶ **Freeze:** each variable becomes a constant.
- ▶ **Remove:** the converter removes the operations that only the training uses.
- ▶ **Fuse:** an activation function becomes a part of the operator before it. A batch normalization becomes a part of the weights of the convolution before it.
- ▶ **Map:** each layer becomes one operator of the LiteRT list.

Key point: The example has six Keras steps and four LiteRT operators. Fewer operators give fewer tensors in RAM.

Check the converted model before you deploy it

```
import numpy as np
from ai_edge_litert.interpreter import Interpreter

interpreter = Interpreter(model_path="model.tflite")
interpreter.allocate_tensors()
inp = interpreter.get_input_details()[0]
out = interpreter.get_output_details()[0]

x = np.random.rand(1, 300).astype("float32")
interpreter.set_tensor(inp["index"], x)
interpreter.invoke()
y_litert = interpreter.get_tensor(out["index"])

y_keras = model(x).numpy()
print(np.abs(y_litert - y_keras).max())      # about 0.00000003
```

- ▶ The LiteRT interpreter runs on the laptop, with the same file that goes to the device.
- ▶ Compare the two outputs for the same input. Then compare the accuracy on the test set. Do this after each conversion step.

Source: Difference: result for the example model with TensorFlow 2.21.0.

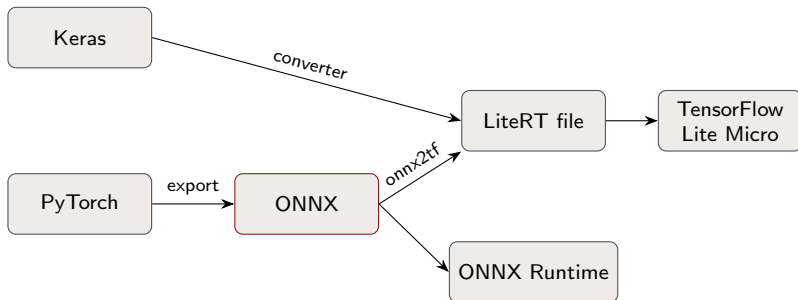
```
python3 tflite_to_header.py model.tflite model.h
```

```
// Model: model.tflite (26732 bytes)
alignas(16) const unsigned char g_model[] = {
    0x1c, 0x00, 0x00, 0x00, 0x54, 0x46, 0x4c, 0x33, 0x14, 0x00, 0x20, 0x00,
    ...
};
const int g_model_len = sizeof(g_model);
```

- ▶ The sketch includes `model.h`. The model is then a part of the program.
- ▶ `const`: the array stays in the flash. With no `const`, the program copies it to RAM at the start.
- ▶ `alignas(16)`: the runtime reads numbers of 4 and 8 bytes at fixed positions. The array must start at a correct address.
- ▶ The bytes 4 to 7 are TFL3: the identifier of a LiteRT file.

Source: The script is in `Labs/hardware/HW-02/`. The command `xxd -i` gives the same bytes.

PyTorch needs one more step: ONNX



- ▶ ONNX is an exchange format for models. Many frameworks write it, and many runtimes read it.
- ▶ A PyTorch model goes to ONNX first. A second tool makes the LiteRT file from the ONNX file.
- ▶ ONNX Runtime runs the ONNX file directly. Day 7 uses it on the Raspberry Pi. It does not run on a microcontroller.

Key point: Each conversion step can change the model. Two steps need two checks.

```
import torch, torch.nn as nn

model = nn.Sequential(
    nn.Linear(300, 20), nn.ReLU(),
    nn.Linear(20, 10), nn.ReLU(),
    nn.Linear(10, 4), nn.Softmax(dim=1),
).eval()

example = torch.rand(1, 300) # fixes the input shape
torch.onnx.export(model, (example,), "model.onnx",
                  input_names=["input"], output_names=["output"])
```

```
onnx2tf -i model.onnx -o saved_model # writes model_float32.tflite
```

- ▶ The export runs the model one time with the example input. The example fixes the shape of each tensor.
- ▶ The ONNX file has the operators Gemm, Relu, Gemm, Relu, Gemm, Softmax.
- ▶ The tool onnx2tf maps them to the LiteRT operators.

Source: Operator list: result of the export with PyTorch 2.14.1 for this model.

The same network in four files

File	Bytes	Storage format	Operators
Keras file (.keras)	43 416	ZIP archive	3 Dense layers
ONNX file from PyTorch	25 818	Protocol Buffers	Gemm, Relu, Softmax
LiteRT file from Keras	26 732	FlatBuffers	FULLY_CONNECTED, SOFTMAX
LiteRT file from ONNX	27 020	FlatBuffers	FULLY_CONNECTED, SOFTMAX

- ▶ All four files hold the same 25 096 bytes of weights. The difference is the description around the weights.
- ▶ The two LiteRT files come from different converters. They are not equal byte for byte, but they calculate the same function.
- ▶ The file size is the flash that the model needs on the device.

Source: Results on the work computer of the course: TensorFlow 2.21.0, PyTorch 2.14.1, onnx2tf 1.29.24.

PyTorch, ONNX: **N C H W**

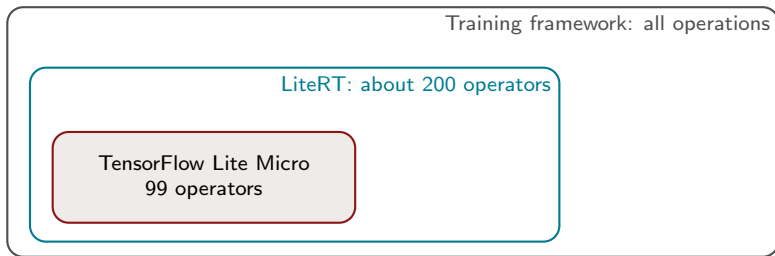
LiteRT: **N H W C**

N: batch, C: channels,
H: height, W: width

- ▶ An image tensor of PyTorch has the shape (1, 3, 96, 96).
- ▶ The same tensor in LiteRT has the shape (1, 96, 96, 3).
- ▶ The converter changes the order of the weights. It can also add **TRANSPOSE** operators, which cost time and RAM.
- ▶ Your code on the device must fill the input in the order of LiteRT.

Key point: A wrong order gives no error. The model runs, and the result is wrong. Compare the outputs with a real input after the conversion.

Operator support decides if a model can run



- ▶ A runtime has one kernel for each operator that it supports. A kernel is the function that calculates the operator.
- ▶ A model can run only if the runtime has a kernel for each of its operators.
- ▶ A smaller runtime has fewer kernels. The common layers are there: convolution, depthwise convolution, fully connected, pooling, softmax.

Source: Numbers: the operator list of TensorFlow 2.21.0, and the file `micro_mutable_op_resolver.h` of the library Chirale_TensorFlowLite 2.0.0.

Cause	Example	Correction
An operator with no kernel	A new activation function, a custom layer	Use a layer from the supported list
A shape that is not fixed	An input of variable length	Fix the input shape before the conversion
Control flow	A loop or a condition that depends on the data	Write the model with no such step
A data type with no kernel	float64, or int8 for an operator that has only a float32 kernel	Change the type, or change the operator

Key point: Read the operator list of the target runtime before you select the layers of the model.

The conversion gives no error. The sketch compiles. On the board, the serial port shows:

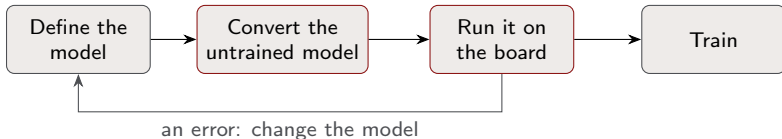
```
Didn't find op for builtin opcode 'SOFTMAX' version '1'.  
Failed to get registration from op code SOFTMAX
```

- ▶ The file is correct. The *program* has no kernel for one operator of the model.
- ▶ With TensorFlow Lite Micro, the program names each operator that it needs. This list is the operator resolver. Part 2 explains it.
- ▶ An operator that the program does not name is not in the firmware. This keeps the firmware small.

Three places for an operator error

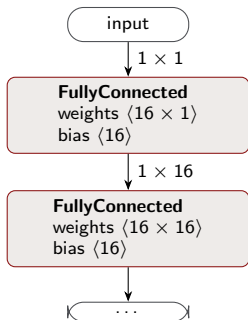
1. At the conversion: the converter has no mapping.
2. At the build: the library has no kernel.
3. On the device: the resolver of the sketch does not name the operator.

Source: Message text: files `op_resolver.cpp` and `micro_interpreter.cpp` of the library Chirale.TensorFlowLite 2.0.0.



- ▶ The conversion and the runtime do not need trained weights. A model with random weights has the same operators, the same tensors, and the same size.
- ▶ So you can test the complete path on the first day: the conversion, the flash, the arena, and the latency.
- ▶ A model that cannot run on the device costs no training time.

Key point: The deployment is not the last step of a project. It is the first test of the model design.



- ▶ Netron is a viewer for model files. It reads LiteRT, ONNX, Keras, and other formats.
- ▶ Open `netron.app` in a browser and select a file. The file stays on your computer.
- ▶ Each box is one operator. Each line is one tensor, with its shape.
- ▶ Click a box: you see the data type of each input, the shape of the weights, and the fused activation function.

Key point: Look at each converted model in Netron. You see in one minute if the operators and the types are the ones that you expect.

Open this file in Netron (`netron.app`):

`Labs/hardware/HW-02/models/hello_world.tflite`

The file is the “hello world” model of TensorFlow Lite Micro. It has 2488 bytes. It calculates an approximation of the sine function.

1. List the operators of the model, in order.
2. What are the shape and the data type of the input and of the output?
3. What is the data type of the weights? What is the data type of the biases?
4. How many parameters does the model have? How many bytes do they need in this file?
5. The file has 2488 bytes. Which part of the file are the parameters?

1. Three operators: FullyConnected, FullyConnected, FullyConnected. The first two have a fused ReLU activation.
2. The input has the shape (1, 1) and the type int8. The output has the same shape and the same type.
3. The weights have the type int8. The biases have the type int32. The model is quantized. Day 4 explains why the bias has more bits.
4. Weights: $16 + 256 + 16 = 288$. Biases: $16 + 16 + 1 = 33$. The model has 321 parameters. They need $288 \times 1 + 33 \times 4 = 420$ bytes.
5. $420/2488 = 17$ percent. For a very small model, the description of the graph is larger than the weights. For the example model of this part, the weights are 94 percent of the file.

Part 1 Deployment formats and conversion

Exercise: list the operators and the tensor types of a LiteRT file in Netron

Part 2 TensorFlow Lite Micro internals

Exercise: estimate the arena size of a small model

Part 3 Features for time series

Exercise: compare the input size of raw windows and of spectral features

- ▶ TensorFlow Lite Micro (TFLM) is a C++ library. It is a part of your firmware.
- ▶ It needs no operating system, no file system, and no `malloc`.
- ▶ It reads the LiteRT file in place, from the flash.
- ▶ It runs the model with a small interpreter.

Sketch for the XIAO

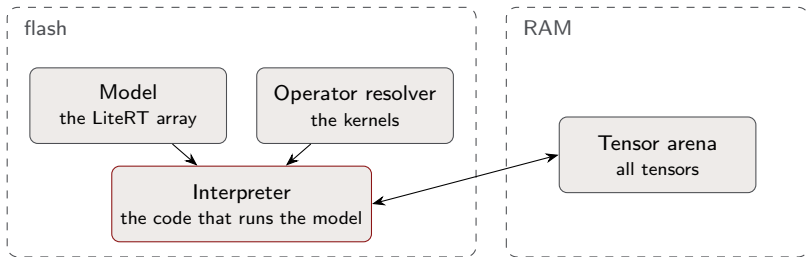
Flash

Empty sketch	271 457
With TFLM, one operator, and a small model	336 157
Difference	64 700

Key point: The runtime with one operator and a model of 2488 bytes adds about 65 KB to the firmware. A training framework has hundreds of megabytes.

Source: Flash in bytes: results of the compiler for the sketch `tflm_hello` of `Labs/hardware/HW-02/`.

The four parts of a TFLM program

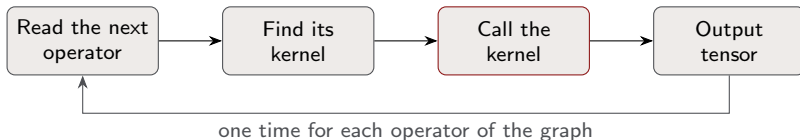


Model The graph and the weights. Constant data in the flash.

Operator resolver The list of the kernels that the firmware holds.

Tensor arena One array in RAM. The program gives it to the interpreter.

Interpreter It reads the graph, and it calls one kernel for each operator.



- ▶ The interpreter does not know the model at build time. It reads the graph from the array when the program starts.
- ▶ A different model needs no different code. You change the array, and perhaps the list of the operators.
- ▶ The cost: one function call for each operator. This cost is small. The kernel does the work.

Key point: The alternative is a compiler that makes C code for one model. That code can be smaller. TFLM selects the interpreter, because one runtime then works for all models.

```
#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"

// The example model: three fully connected operators and one softmax.
// These are two operator types.
static tflite::MicroMutableOpResolver<2> resolver;

resolver.AddFullyConnected();
resolver.AddSoftmax();
```

- ▶ The number in <2> is the number of operator types, not the number of operators.
- ▶ Each Add...() call puts one kernel in the firmware. The linker removes each kernel that the program does not name.
- ▶ Netron shows the operator types of your model.

Key point: The model selects the operators. An operator with no Add...() call gives the error of Part 1 on the device.

The cost of all operators

One operator  64 700 bytes

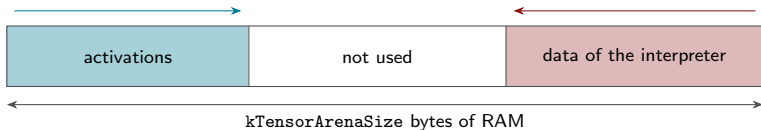
All operators  286 080 bytes

- ▶ The library has a resolver with all operators. It is easy to use: each model runs.
- ▶ It adds 221 380 bytes to the firmware in comparison with one operator.
- ▶ On the XIAO, this is 7 percent of the program space. On a microcontroller with 1 MB of flash, it is 21 percent.

Key point: Name only the operators of your model. The flash that you save is space for a larger model.

Source: Results of the compiler for the sketch `tflm_hello` with `MicroMutableOpResolver<1>` and with all operators. The bars show the flash that TFLM adds to an empty sketch.

```
constexpr int kTensorArenaSize = 4 * 1024;  
alignas(16) uint8_t tensor_arena[kTensorArenaSize];
```



- ▶ The arena is one array. The program defines it. The linker gives it a fixed place in RAM.
- ▶ From one end: the activations, that is the input, the output, and each tensor between two operators.
- ▶ From the other end: the data that the interpreter keeps for each tensor and for each operator.
- ▶ The weights are not in the arena. They stay in the flash.

With `malloc`

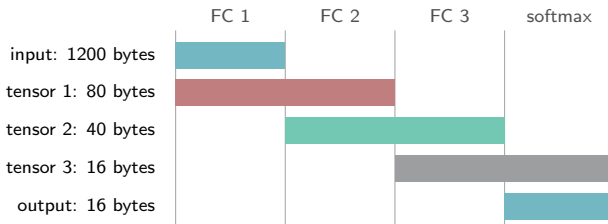
- ▶ The program asks for memory when it needs it.
- ▶ Free blocks become small pieces. A large request then fails, with enough free memory in total.
- ▶ The failure can come after days of operation.
- ▶ A device that operates for years with no reset needs memory use that does not change.

With the arena

- ▶ The interpreter takes all its memory one time, in `AllocateTensors()`.
- ▶ After that call, it takes no more memory.
- ▶ If the arena is too small, the program fails at the start, on your desk.

Key point: If `AllocateTensors()` returns with no error, an inference cannot fail later because of memory.

The static memory plan



- ▶ Each activation has a lifetime: from the operator that writes it to the last operator that reads it.
- ▶ Two tensors with lifetimes that do not overlap can use the same bytes. The input and tensor 2 are never in use at the same time.
- ▶ The interpreter calculates this plan one time, in `AllocateTensors()`. Each tensor gets a fixed position in the arena.

Key point: The plan is static: the positions do not change between two inferences.

Operator	Tensors in use	Input	Output	Sum in bytes
FC 1	input, tensor 1	1200	80	1280
FC 2	tensor 1, tensor 2	80	40	120
FC 3	tensor 2, tensor 3	40	16	56
Softmax	tensor 3, output	16	16	32

- ▶ The example model has 300, 20, 10, and 4 values of `float32`: 4 bytes for each value.
- ▶ The activations need 1280 bytes at the peak. With no reuse, they need $1200 + 80 + 40 + 16 + 16 = 1352$ bytes.
- ▶ This is the rule of Day 1: the peak is the largest sum of the tensors that are in use at the same time.

Key point: For a chain of layers, the peak is the layer with the largest sum of input and output. In most models, this is the first layer.

The arena holds the activations and the data of the interpreter. The second part depends on the number of tensors and operators. No simple formula gives it.

```
if (interpreter->AllocateTensors() != kTfLiteOk) {  
    stop("ERROR: AllocateTensors() failed. The arena is too small.");  
}  
Serial.print("Arena used (bytes): ");  
Serial.println(interpreter->arena_used_bytes());
```

1. Calculate the peak of the activations. Start with an arena that is much larger.
2. Run the sketch. Read the value of `arena_used_bytes()`.
3. Set the arena size to this value plus a small margin.

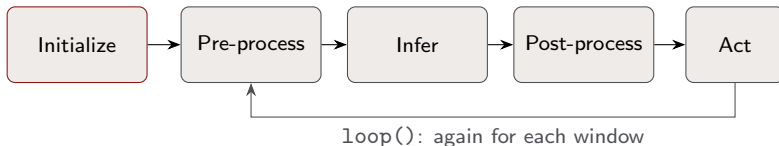
When the arena is too small, the library prints a message of this form:

```
Failed to resize buffer. Requested: ..., available ..., missing: ...
```

Source: Code: sketch tf1m_hello. Message text: library Chirale_TensorFlowLite 2.0.0.

The life of a TFLM program

setup(): one time



Initialize Map the model, make the resolver and the interpreter, allocate the tensors.

Pre-process Read the sensor, fill the window, calculate the features, write the input tensor.

Infer Call `Invoke()`.

Post-process Read the output tensor, select the class, filter the result.

Act Show the class on the display, switch a pin, send a message.

Initialize: one time in `setup()`

```
#include <Chirale_TensorFlowLite.h>
#include "model.h" // g_model, from the converter
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"
#include "tensorflow/lite/schema/schema_generated.h"

constexpr int kTensorArenaSize = 4 * 1024;
alignas(16) uint8_t tensor_arena[kTensorArenaSize];
tflite::MicroInterpreter* interpreter = nullptr;

void setup() {
    const tflite::Model* model = tflite::GetModel(g_model); // no copy

    static tflite::MicroMutableOpResolver<2> resolver;
    resolver.AddFullyConnected();
    resolver.AddSoftmax();

    static tflite::MicroInterpreter static_interpreter(
        model, resolver, tensor_arena, kTensorArenaSize);
    interpreter = &static_interpreter;
    interpreter->AllocateTensors(); // the memory plan
}
```

Source: The code follows the sketch `tflm_hello`, which adapts the example `hello_world` of the library `Chirale_TensorFlowLite`. The frame leaves out the error checks.

```
TfLiteTensor* input = interpreter->input(0);
TfLiteTensor* output = interpreter->output(0);

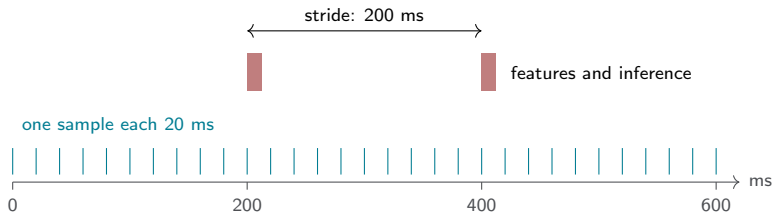
// Pre-process: copy the window into the input tensor.
for (int i = 0; i < 300; i++) {
    input->data.f[i] = window[i];
}

// Infer, and measure the time.
const unsigned long t0 = micros();
interpreter->Invoke();
const unsigned long latency_us = micros() - t0;

// Post-process: find the class with the largest probability.
int best = 0;
for (int k = 1; k < 4; k++) {
    if (output->data.f[k] > output->data.f[best]) best = k;
}
```

- ▶ data.f is for a float32 tensor. A quantized model uses data.int8 (Day 4).
- ▶ The input tensor and the output tensor are places in the arena.

The loop of the motion classifier



- ▶ The sampling continues at 50 Hz. A ring buffer holds the newest 2 s: the window.
- ▶ After each stride of 200 ms, the program calculates the features of the window and runs the model.
- ▶ This work must be complete before the next sample: in less than 20 ms. If not, the sampling rate changes, or a timer interrupt must read the sensor.

Key point: The latency of the model is one part of a time budget.

Class	Window 1	Window 2
idle	0.02	0.30
terrestrial	0.05	0.25
lift	0.90	0.28
maritime	0.03	0.17

Window 1: a clear result.

Window 2: the largest value is 0.30.

This is not a decision.

- ▶ **Threshold:** accept a class only if its probability is above a limit, for example 0.6. If not, show “uncertain”.
- ▶ **Filter in time:** use the class that wins in three of the last five windows.
- ▶ **Delay:** the filter makes the output stable. It also makes it later.

Key point: The model gives numbers. The post-processing makes the decision. Its settings are part of the system design.

Kernels	Processor	Method
Reference	All processors	Portable C++. Simple and slow.
CMSIS-NN	Arm Cortex-M	Instructions of the processor that calculate on some values at the same time
ESP-NN	ESP32-S3 and other chips of Espressif	The vector instructions of the ESP32-S3

- ▶ A model spends almost all its time in the kernels. An optimized kernel gives the same result in less time. The model file does not change.
- ▶ Most optimized kernels are for `int8` models. This is one more reason for the quantization of Day 4.

In the lab

Your own sketch uses a library with reference kernels on the ESP32-S3. The library of Edge Impulse uses ESP-NN. Part D compares the two latencies.

Three numbers that you measure on the device

Number	Where it comes from	What changes it
Flash	The line "Sketch uses ... bytes" of the build	The size of the model file, the number of operator types
Arena	The call <code>arena_used_bytes()</code> of the interpreter	The largest activations, the number of tensors
Latency	<code>micros()</code> before and after <code>Invoke()</code>	The operations of the model, the kernels, the clock frequency

- ▶ These are the three budgets of Day 1 on a real device: parameters, activation memory, and operations.
- ▶ Measure the latency many times. Report the median and the maximum, not one value.
- ▶ In the lab, you measure the three numbers with PSRAM and with no PSRAM.

Key point: A number from the laptop is an estimate. Only the number from the board is a result.

Exercise: estimate the arena size

Model A. The example model of this part, converted to int8: 300 inputs, then 20, 10, and 4 values. Each value needs 1 byte.

Model B. A small network for images, in int8:

Layer	Input	Output
Convolution, stride 2, 8 filters	$96 \times 96 \times 1$	$48 \times 48 \times 8$
Convolution, stride 2, 16 filters	$48 \times 48 \times 8$	$24 \times 24 \times 16$
Global average pooling	$24 \times 24 \times 16$	16
Fully connected	16	4

1. Calculate the peak of the activations of each model, in bytes.
2. Which layer gives the peak?
3. Which arena size do you try first for each model?
4. Model B has 1316 parameters. Model A has 6274. Which model needs more flash? Which model needs more RAM?

1. **Model A:** $300 + 20 = 320$ bytes. **Model B:** $96 \times 96 \times 1 = 9216$ and $48 \times 48 \times 8 = 18432$. The sum is 27 648 bytes.
2. The first layer in the two models. In model B, the second layer needs $18432 + 9216 = 27648$ bytes also: the same sum.
3. The peak is the minimum. The arena also holds the data of the interpreter. Start with much more: 4 KB for model A, and 40 KB for model B. Then read `arena_used_bytes()` and decrease the size.
4. Model A needs more flash: 6274 parameters against 1316. Model B needs more RAM: 27 648 bytes of activations against 320.

The parameters decide the flash. The activations decide the RAM. A model with few parameters can need much RAM.

Part 1 Deployment formats and conversion

Exercise: list the operators and the tensor types of a LiteRT file in Netron

Part 2 TensorFlow Lite Micro internals

Exercise: estimate the arena size of a small model

Part 3 Features for time series

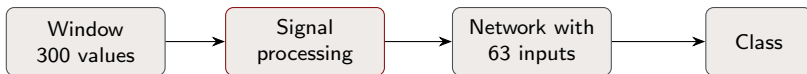
Exercise: compare the input size of raw windows and of spectral features

Two ways to give a window to a model

Raw window



Features



- ▶ A feature is one number that describes a property of the window: how strong the motion is, how fast it is.
- ▶ Signal processing calculates the features with fixed formulas. It has no trained weights.
- ▶ The network then learns only the step from the features to the class.

Key point: The question of this part: which input gives the smaller model and the smaller arena for the same task?

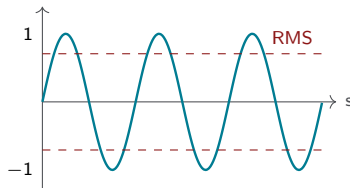
Feature	What it measures	In the motion classes
Mean	The constant part of the signal	Gravity: the angle of the kit
RMS	The energy of the signal	Near 0 for “idle”, large for a strong motion
Minimum, maximum	The range of the values	The size of the motion
Skewness	If the values are symmetric around the mean	A motion with a fast and a slow direction
Kurtosis	If the signal has short, large peaks	A shock

- ▶ Each feature is one number for each axis. The order of the samples in the window has no effect on these five features.
- ▶ You calculate them with one loop over the samples.

Source: Feature list: *Machine Learning Systems*, hardware kits, “DSP Spectral Features”.

$$x_{\text{RMS}} = \sqrt{\frac{1}{n} (x_1^2 + x_2^2 + \cdots + x_n^2)}$$

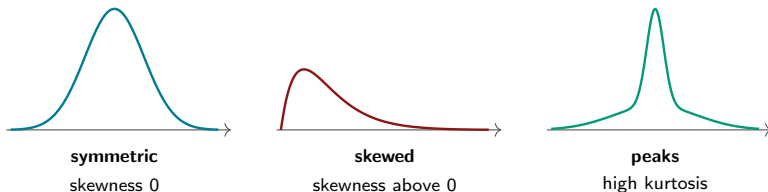
- ▶ RMS: root mean square.
- ▶ Subtract the mean of the window first. If not, gravity decides the value of the z axis.
- ▶ After this step, the RMS is equal to the standard deviation.



A sine wave with the amplitude 1 has an RMS of $1/\sqrt{2} = 0.707$.

Key point: The RMS of three axes gives three numbers. They separate “idle” from motion, and a horizontal motion from a vertical motion.

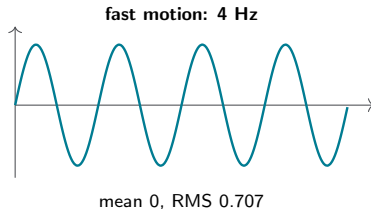
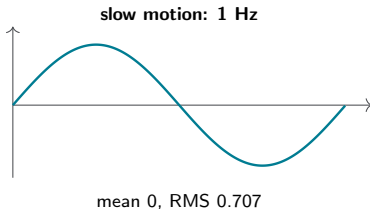
Skewness and kurtosis: the form of the distribution



- ▶ Each curve shows how often each value occurs in a window.
- ▶ **Skewness** measures the symmetry. A motion that goes fast in one direction and slowly back gives a skewed distribution.
- ▶ **Kurtosis** measures the weight of the rare, large values. A signal that is quiet with some shocks has a high kurtosis.
- ▶ The RMS says how large the signal is. These two features say which form it has.

Source: Definitions: *Machine Learning Systems*, hardware kits, "DSP Spectral Features".

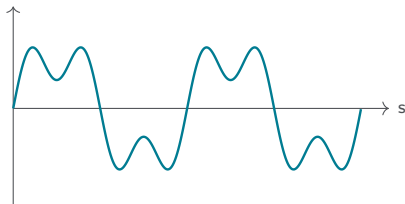
The statistics do not see the speed



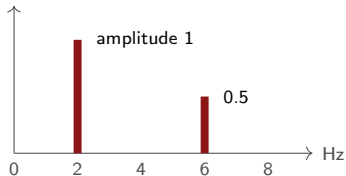
- ▶ The two signals have the same mean, the same RMS, the same minimum, and the same maximum.
- ▶ The statistical features are equal. A model with these features cannot separate the two motions.
- ▶ The difference is the frequency. You need a feature that measures it.

Key point: The time domain says how large the signal is. The frequency domain says how fast it changes.

The frequency domain



time domain



frequency domain

- ▶ Each signal is a sum of sine waves with different frequencies.
- ▶ The signal on the left is the sum of two sine waves: 2 Hz with the amplitude 1, and 6 Hz with the amplitude 0.5.
- ▶ The spectrum on the right shows the same signal: one bar for each frequency.
- ▶ The Fourier transform calculates the spectrum from the samples. The FFT is a fast method for it.

- ▶ The FFT takes N samples. N is the FFT length, a power of two.
- ▶ It gives $N/2 + 1$ values. Each value is one frequency bin.
- ▶ The width of one bin is f_s/N .
- ▶ The last bin is at $f_s/2$, the Nyquist frequency of Day 2.

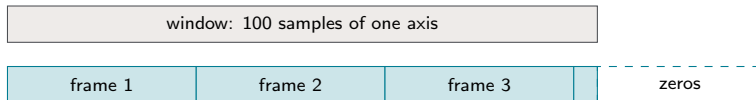
The motion lab

$f_s = 50$ Hz and $N = 32$:

- ▶ 17 bins, from 0 to 25 Hz
- ▶ bin width $50/32 = 1.56$ Hz

FFT length	Bins	Bin width at 50 Hz	Effect
16	9	3.13 Hz	Few features, low resolution
32	17	1.56 Hz	The setting of the kit lab
64	33	0.78 Hz	Many features, high resolution

Key point: The FFT length is a design decision: more bins separate frequencies better, and they make the model input larger.



FFT of each frame, power of each bin, then the maximum of each bin over the frames

- ▶ The window has 100 samples. The FFT length is 32. So the block cuts the window into frames of 32 samples.
- ▶ The power of a bin is the square of its FFT value, divided by the FFT length.
- ▶ For each bin, the block keeps the largest power over the frames.
- ▶ The bin at 0 Hz is not a feature: the block removed the mean before. The result is 16 power values for each axis.

Source: Method of the spectral analysis block of Edge Impulse: *Machine Learning Systems*, hardware kits, "DSP Spectral Features".

The feature block of the kit lab

Group	Features for each axis	Number
Statistics in the time domain	RMS, skewness, kurtosis	3
Statistics of the spectrum	Skewness and kurtosis of the power values	2
Spectral power	One value for each bin from 1 to 16	16
Sum for one axis		21
Sum for three axes		63

- ▶ The block first removes the mean of each axis in the window.
- ▶ The 63 features are the input of the classifier: 63 neurons in the input layer, then 20, 10, and 4.
- ▶ Edge Impulse Studio calculates the same features. You compare the two paths in Part D of the lab.

Source: *Machine Learning Systems*, XIAOML Kit, “Motion Classification and Anomaly Detection”, and “DSP Spectral Features”.

```
import numpy as np

def spectral_power(x, nfft=32):
    """Largest power of each frequency bin over the frames of the window."""
    power = np.zeros(nfft // 2 + 1)
    for start in range(0, len(x), nfft):
        frame = x[start:start + nfft] # the last frame is short
        spectrum = np.abs(np.fft.rfft(frame, nfft)) # rfft adds zeros
        power = np.maximum(power, spectrum ** 2 / nfft)
    return power[1:] # no bin at 0 Hz

x = window[:, 0] # 100 samples of the x axis
x = x - x.mean() # remove gravity and the offset
rms = np.sqrt(np.mean(x ** 2))
power = spectral_power(x) # 16 values
```

- ▶ The training notebook calculates the features of each window with this code.
- ▶ The sketch on the board calculates the same features in C++. The two results must be equal for the same window.

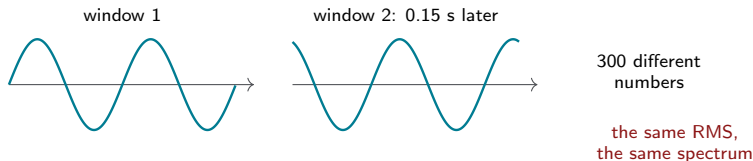
Source: The function follows `welch_max_hold` of “DSP Spectral Features” in *Machine Learning Systems*.

The model becomes smaller

	Raw window	Features
Input values	300	63
Parameters of the first layer (20 neurons)	6020	1280
Parameters of the complete model	6274	1534
Weights in float32, in bytes	25 096	6136
Peak of the activations, in bytes	1280	332

- ▶ The two models have the same hidden layers: 20 and 10 neurons, and 4 outputs.
- ▶ In a fully connected network, the first layer holds almost all parameters. Its size is the number of inputs times the number of neurons.
- ▶ The signal processing is not free. It needs code in the flash and time on the processor. You measure the two in the lab.

Why features need less model: invariance



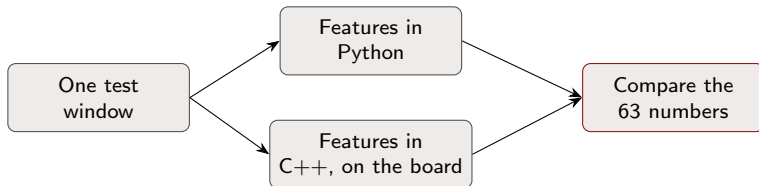
- ▶ The two windows hold the same motion. The second window starts later.
- ▶ As raw input, the two windows are two different vectors. The network must learn that they are the same class, for each possible shift.
- ▶ The RMS and the spectral power do not change with the shift. The features hold what the network must learn if not.

Key point: A feature that removes a variation saves the parameters and the training data that the network needs to learn that variation.

The cost moves to the signal processing

	Raw window	Features
Flash	A larger model	A smaller model, and the code of the features
RAM	A larger arena	A smaller arena, and buffers for the FFT
Latency	The inference only	The features, then a shorter inference
Design	No knowledge of the signal is necessary	You must select the features
Data	More training data is necessary	Less training data is necessary

- ▶ The pre-processing is a part of the latency. Measure the time of the features and the time of `Invoke()` separately.
- ▶ An FFT of 32 points is a small calculation. For a microphone at 16 kHz, the signal processing can need more time than the model.



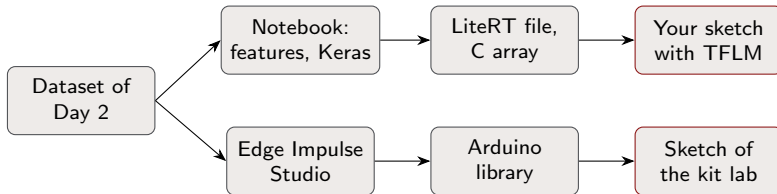
- ▶ The network learned from the features of the Python code. On the board, C++ code calculates the features.
- ▶ A small difference changes the input of the model: a different FFT length, no mean removal, a different unit.
- ▶ This is the training-serving skew of Day 2. It gives no error message.

Key point: Test the feature code of the board with a window that has a known result. Do this before you test the model.

Sensor	Usual input of the model	Reason
IMU	Statistical and spectral features, with a small fully connected network	The physics of the motion is known. The model is very small.
Microphone	A spectrogram or MFCC features, with a small convolutional network (Day 5)	A raw second of audio has 16 000 values. The spectrum has much less.
Camera	The pixels, with a convolutional network (Day 5)	No simple formula describes an object. The network learns its filters.

- ▶ A convolution layer on a raw time series is a set of filters that the training finds. It can replace the FFT, with more parameters and more data.
- ▶ When you know which property of the signal separates the classes, calculate that property directly.

The lab: two paths to the same classifier



- ▶ **Part A:** calculate the features of your windows and train a small classifier.
- ▶ **Part B:** convert the model, write the inference code, and show the class on the display.
- ▶ **Part C:** measure the arena, the flash, and the latency, with PSRAM and with no PSRAM.
- ▶ **Part D:** train the same task in Edge Impulse Studio, and compare the two results in one table.

Key point: The two paths use the same data and the same type of features. The difference is who writes the code, and which kernels run.

A motion classifier uses a window of 2 s with 3 axes. The first layer of the network has 20 neurons. The feature block gives 21 features for each axis.

1. The sampling rate is 50 Hz. How many values has the raw window? How many features has the feature block?
2. Calculate the parameters of the first layer for the two inputs.
3. The team changes the sampling rate to 100 Hz. The window stays at 2 s, and the FFT length stays at 32. Calculate the two input sizes and the parameters of the first layer again.
4. At 100 Hz with an FFT length of 32, what is the width of one frequency bin? What is the highest frequency of the spectrum?
5. Which input do you select for a device with 2 KB of free RAM? Give the reason.

Answer: raw windows and spectral features

1. Raw window: $2 \times 50 \times 3 = 300$ values. Features: $21 \times 3 = 63$.
2. Raw window: $300 \times 20 + 20 = 6020$. Features: $63 \times 20 + 20 = 1280$.
3. Raw window: $2 \times 100 \times 3 = 600$ values and $600 \times 20 + 20 = 12\,020$ parameters. Features: 63 values and 1280 parameters, with no change. The number of features depends on the FFT length, not on the sampling rate.
4. Bin width: $100/32 = 3.13$ Hz. Highest frequency: 50 Hz. The same number of bins now covers two times the frequency range, so each bin is two times as wide.
5. The features. With `float32`, the raw window at 50 Hz needs 1200 bytes for the input and 80 bytes for the first output: 1280 bytes of the 2 KB, with no space for the data of the interpreter. The features need $63 \times 4 + 80 = 332$ bytes.

- ▶ A LiteRT file is a FlatBuffer: the graph, the operators, and the weights in one file that the device reads with no parsing step.
- ▶ A conversion fails when the target runtime has no kernel for an operator of the model.
- ▶ TensorFlow Lite Micro is an interpreter with no dynamic allocation. All tensors are in one arena that the program gives.
- ▶ Features from signal processing make the input smaller. A smaller input needs a smaller model and a smaller arena.

Key point: Three numbers describe a deployed model: the flash use, the arena size, and the latency. Measure all three on the device.

Book chapters

- ▶ *Machine Learning Systems* (mlsysbook.ai), Volume I: chapter 7 “ML Frameworks”, section “Deployment Targets”.
- ▶ *Machine Learning Systems*, hardware kits, XIAOML Kit: “Motion Classification and Anomaly Detection”, and the shared chapter “DSP Spectral Features”.
- ▶ *XIAO: Big Power, Small Board*, chapter 4.2 “Anomaly Detection & Motion Classification”.

Papers

- ▶ “TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems”, David et al., 2021.

Tools

- ▶ LiteRT documentation: <https://ai.google.dev/edge/litert>
- ▶ Netron, a viewer for model files: <https://netron.app>
- ▶ ONNX: <https://onnx.ai>

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *TinyML Made Easy: XIAO ESP32S3* by Marcelo Rovai. github.com/Mjrovai/XIAO-ESP32S3-Sense, Apache-2.0.
- ▶ *XIAO: Big Power, Small Board* by Lei Feng and Marcelo Rovai. github.com/Mjrovai/XIAO_Big_Power_Small_Board-ebook, GPL-3.0.